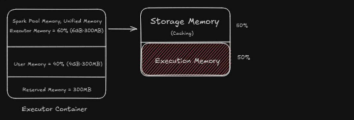
DATABRICKS and PYSPARK Interview Guide

Ques1: You have a Spark job that needs to process 10GB of data. How would you decide the number of cores and memory per executor for this job? Explain your reasoning.

Solution :



10GB

128 MB = 1 Partition
10 GB Data = 80 Partitions

→ 80 cores

→ 10 Executors

1 Executor has 8 cores (assumption)

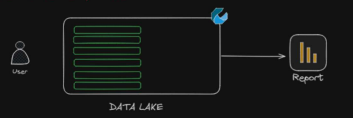
Memory Allocation

8 Cores = 8 Partitions (128MB each) = 1GB

→ 3.5GB Per Executor (3.5X)
(Based on the Spark Pool Memory Allocation)

Ques2: You are working as a Data Engineer in a company where daily ingestion jobs write data into a Data Lake (Delta/Parquet). Over time, the storage is filled with thousands of small files, causing slow query performance and high job runtimes.

How would you tackle the small file problem:
- While writing the data into the Data Lake?
- After the data is already written?

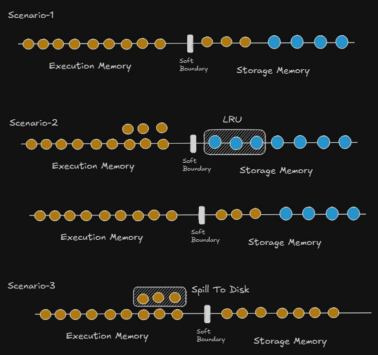


OPTIMIZE COMMAND - DELTA LAKE

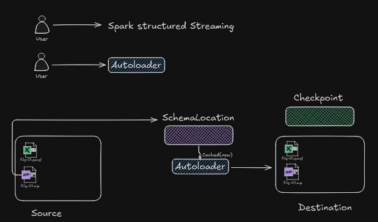
```
OPTIMIZE <table_name>;
```


```
optimizeWrite
  df.write.format("delta")\
    .option("optimizeWrite",True)\
    .mode("append")\
    .....
```

Ques3: How does Spark's Unified Memory Manager allocate memory between execution memory and storage memory in this scenario?
How does Spark reallocate memory dynamically when execution needs more memory, and what happens to cached data during this process?
- Jobs are slowing down
- Cached Dataframes are evicted

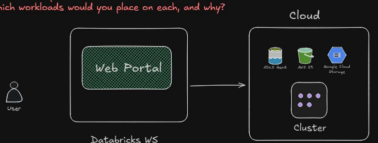


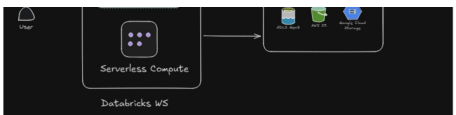
Ques4: Your team receives event data as files landing continuously in cloud storage. The source team sometimes adds new columns or changes the schema without prior notice.
You need to process these files in near-real time and load them into a Delta table without breaking the pipeline.
How would you design this streaming solution to handle schema changes safely?
What Spark/Databricks features would you use?



Ques5 : Your team is building a data platform on Databricks.
- Daily ETL jobs run once a day and finish in 20–30 minutes
- Ad-hoc analysis is done by data engineers and analysts throughout the day
- Cost optimization and cluster stability are important

How would the architecture differ if you use Serverless Compute vs All-Purpose Compute for this setup?
Which workloads would you place on each, and why?





Ques6: You're working as a Data Engineer for an e-commerce company.
The business team tracks a daily revenue KPI in Databricks using a Delta table.

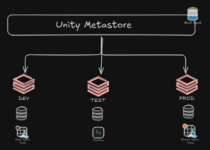
If daily revenue drops by more than 10% compared to the previous day, the business stakeholders should get an alert immediately on email or Slack.
How would you design and implement alerts in Databricks to notify the respective stakeholders?

Ques7 : Your finance and business teams use Databricks for analysis, but they are not comfortable editing SQL queries.
They often ask you to run the same SQL logic again and again, just with different dates, countries, or product names.

Question:
How would you design re-usable SQL queries in Databricks so that non-technical users can run the analysis by just changing inputs, without touching the core SQL logic?

Ques8: Your organization is using multiple Databricks workspaces (dev, test, prod) across different teams. Data is stored in a centralized cloud storage account, and you want:
- Fine-grained access control (table, column, row level)
- A single governance layer across all workspaces
- Central auditing and lineage

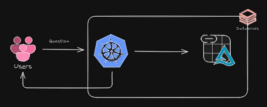
How would you design the architecture using Unity Catalog in a modern Databricks workspace?
Explain the key components involved (metastore, catalogs, schemas, storage, and access control) and how they interact.



Ques9: Your team uses a Serverless SQL Warehouse for BI dashboards.
During business hours, hundreds of users hit the dashboard and queries slow down. At night, usage drops to almost zero.

Which compute type would you choose for the SQL Warehouse, and how does it scale automatically to handle this pattern?

Ques10: Your company has a Databricks Lakehouse with Delta tables for sales, customers, and transactions.
Business users (non-technical) want to ask questions in plain English like:
- "What were last quarter's top-selling products in Canada?"



Ques11: You work as a Data Engineer at an airline company. The business team wants to understand how flight ticket sales are trending over time to plan pricing and promotions.
Using PySpark, how would you analyze the trend of flight ticket sales over time (daily or monthly)?

Ques12: You work as a Data Engineer for an airline analytics team. The business wants to identify the top 5 airports based on total flight ticket sales for the last few months.
How would you calculate the top 5 airports by total sales revenue?
Explain the logic.

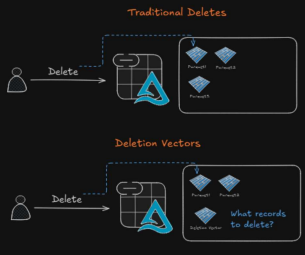
Ques13: Every booking generates a sale_amount for a flight. Management wants to track monthly performance trends, so they ask you to calculate a running total of sales for each flight, but the total should reset at the start of every month.

Ques14: You are assigned to find the revenue generated by each city so far and tag those cities as high mid low based on the revenue buckets.
- Less than 1000 (low)
- Between 1000 - 20,000 (mid)
- More than 10,000 (high)

Ques15: Multiple source systems send data with different schemas, but all of them contain some common numeric columns that need aggregation.
The business frequently changes:
- Which columns to aggregate
- Which aggregation functions to apply (sum, avg, max, count, etc.)
- Hard-coding logic every time is not acceptable.

Ques16: You have a large Delta table in Databricks (billions of rows) used for analytics.
Due to data privacy rules, you need to delete a small subset of records frequently (for example, user-requested deletes). Rewriting the entire data files every time is slow and expensive.

How do Deletion Vectors in Databricks help in this scenario?
What do they do under the hood, and how do they improve performance compared to traditional deletes?



Ques17: You're working on a Delta table in Databricks that stores daily transaction data.
While fixing a bug, you accidentally ran a DELETE statement that removed a large number of valid records from the table.
The job completed successfully, but later you realized the mistake.

How would you recover the table to its previous state using Delta Lake?

Ques18: You have a Delta table with 2–3 TB data where new records are continuously

Earlier it was partitioned by date, but now queries filter on customer_id and region, and partitions are causing too many small files.

How would Liquid Clustering help here, and why is it better than traditional partitioning?